AED2 - Atividade Prática - Aula 2

Aluno: João Marcelo Franca Batista

1) Ordenação quadrática: Bubble Sort x Insertion Sort

Especificações da máquina usada no teste

Processador: Intel(R) Core(TM) i5-1035G1 @ 1.00GHz (4 núcleos / 8 threads)
Memória RAM: 8 GB
Sistema operacional: Windows 11 Home Single Language (64-bit)
Compilador: gcc 13.2.0 (MinGW-w64 / MSYS2), sem otimização (-O0)

Tempo de execução

Cada vetor tem 10.000 elementos, gerado com semente fixa (srand(42)). Os vetores 'crescentes' usam os mesmos valores dos vetores aleatórios correspondentes, apenas em ordem crescente, para que a comparação isole o efeito da ordem de entrada. O tempo foi medido com clock_gettime(CLOCK_MONOTONIC), que tem precisão de nanossegundos.
Inteiros aleatórios: Bubble Sort: 0.542060 s | Insertion Sort: 0.132646 s
Inteiros crescentes: Bubble Sort: 0.261109 s | Insertion Sort: 0.000074 s
Caracteres aleatórios: Bubble Sort: 0.380623 s | Insertion Sort: 0.108294 s
Caracteres crescentes: Bubble Sort: 0.209155 s | Insertion Sort: 0.000037 s

Resultado: o Insertion Sort foi mais rápido em todos os cenários testados, com destaque para os vetores já crescentes, onde ele é milhares de vezes mais rápido que o Bubble Sort.

Análise de complexidade de tempo

Notação usada: $O(f(n))$ para o pior caso, $\Theta(f(n))$ para o caso médio e $\Omega(f(n))$ para o melhor caso.
O Bubble Sort compara pares de elementos vizinhos e troca quando estão fora de ordem, repetindo isso em vários passos pelo vetor inteiro. Do jeito que foi implementado, ele sempre faz o mesmo número de comparações, não importa se o vetor já está ordenado ou não. Por isso sua complexidade de tempo é a mesma nos três casos: $O(n^2)$ no pior caso, $\Theta(n^2)$ no caso médio e $\Omega(n^2)$ no melhor caso.
O Insertion Sort percorre o vetor e vai inserindo cada elemento na posição certa entre os que já foram organizados. Se o vetor já está quase ordenado, ele quase não precisa mover nada, o que o torna muito mais rápido nesse caso. Por isso sua complexidade de tempo é $O(n^2)$ no pior caso e $\Theta(n^2)$ no caso médio (vetor aleatório ou decrescente), mas cai para $\Omega(n)$ no melhor caso (vetor já crescente).
Isso explica os resultados: nos vetores aleatórios os dois algoritmos estão na ordem de $\Theta(n^2)$, mas o Insertion Sort ainda é mais rápido porque faz menos operações por comparação. Já nos vetores crescentes, o Insertion Sort cai para o seu melhor caso $\Omega(n)$, enquanto o Bubble Sort continua em $\Omega(n^2)$ mesmo no melhor caso, o que faz a diferença de tempo ser gigante.